

PureScript backends: a comparison

Seed material for a future comparison site covering the whole backend family. This table covers the five backends we have local checkouts and working knowledge of: the reference JS backend in its two habitats (browser, Node), `purel` (Erlang/BEAM), `purescript-julia` (Jurist), and `katsujukou`'s `purescript-backend-wasm` (Wasm GC). To be added as the site project spins up: `purescript-native` (Go, cloned at `purescript-backends/purescript-native`), the from-scratch Python backend (`purescript-python-new`), `purs-backend-es` (the optimizing JS backend — same habitat as JS, different generator, which makes it a useful control column), and Fabrizio's upcoming Racket backend.

A note on framing: "backend" mixes two axes — *code generator* and *runtime habitat*. Browser and Node share `purs`'s JS code generator and differ only in runtime and FFI ecosystem; `purel` and Jurist are separate code generators consuming CoreFn. The table keeps all four columns because the *experienced* differences (FFI surface, concurrency, what programs make sense) follow the habitat, not just the generator.

The table

	JS (browser)	JS (Node)	<code>purel</code> (BEAM)	Jurist (Julia)	Wasm (katsujukou)
Code generator	<code>purs</code> itself (CoreFn → CoreImp → optimized ES modules)	same	standalone <code>purel</code> , consumes CoreFn JSON	standalone <code>purejl</code> (Haskell), consumes CoreFn JSON	standalone compiler written in PureScript , consumes CoreFn JSON + <code>externs.cbc</code> Wasm GC module via Binaryen
Status	reference, definitional	reference	production-mature (id3as media streaming); maintained package sets	experimental, weekend-shaped; core libs working	experimental, advancing fast (25 ADRs benchmarks); agent-assisted development
Semantics vs reference	— is the reference	—	diverges by design (Int, strings); own test suites	422/426 differential tests byte-identical ; 4 documented divergences	same-source benchmarks vs <code>purs</code> JS and <code>purel</code> backend-es; Int is <code>i32</code> (JS-aligned)
Functions	curried unary; optimizer inlines/uncurries hot paths	same	curried funs; arity-optimized top-level variants	curried unary closures, <code>(f)(x)(y)</code>	uniform <code>eqref</code> calling convention; partial application supported; aggressive inlining order specialization
ADT values	constructor functions + <code>instanceOf</code> dispatch, fields <code>value0...</code>	same	tagged tuples, atom tag: <code>{just, X}</code>	tag-tuples: <code>("Just", x)</code> , tag at <code>[1]</code>	struct <i>subtypes</i> of a tag-only <code>\$Data</code> block; like ADTs as unboxed <code>i31</code> ; scalar fields as struct
Newtypes	erased	erased	erased	erased	erased
Records	JS objects	same	Erlang maps (atom keys)	<code>Dict{String,Any} + merge</code>	<code>\$Rec</code> struct of parallel arrays: interned values; polymorphic update (ADR-002)
Typeclass dictionaries	JS objects; common instances inlined by the optimizer	same	maps	<code>Dict{String,Any}</code> keyed by member name	eliminated — positional specialization recursive instance groups handled
Int	double wrapped <code> 0 → int32</code>	same	bignum (arbitrary precision)	Int64 (Bits/pow do apply JS <code>ToInt32</code>)	i32 (true 32-bit, JS-aligned); boxed <code>\$Int32</code> unboxed to raw <code>i32</code> by the optimizer
Number	IEEE double	same	float (double)	Float64; <code>show</code> reproduces JS <code>toString</code> placement rules	f64 (boxed <code>\$Num</code> , unboxed by the optimizer)
Strings	UTF-16 code units	same	UTF-8 binaries	UTF-8 <code>String</code> ; CodeUnits API is codepoint-based (BMP-identical)	UTF-8 bytes in a Wasm GC array
Effect	nullary thunk; MagicDo collapses binds	same	nullary funs	nullary thunk	native lowering — collapses to constant loops; whole-program purity analysis (0015/0018/0019)

	JS (browser)	JS (Node)	purel (BEAM)	Jurist (Julia)	Wasm (katsujukou)
TCO	purs optimizer: self-tail-calls → <i>while</i> loops	same	native BEAM TCO, including mutual recursion	trampoline mirroring purs's optimizer: self-tail-calls → dispatch loops (verified 10%)	tail-call elimination in codegen; survive recursion where both JS backends over (bintreeBfs)
Mutual recursion (unbounded)	MonadRec idiom	same	free (native TCO)	MonadRec idiom (matches JS)	TCE in codegen (extent: see ADRs)
Lazy/recursive bindings	runtime lazy thunks	same	similar runtime support	<i>_runtime_lazy</i> thunks, smart Rec partition	recursive let-bindings supported; pure globals instantiated at start (ADR-000)
FFI unit	<i>.js</i> ES module per PS module	same	<i>.erl</i> module per PS module	<i>Module_foreign.jl</i> included <i>inside</i> the generated module	curated <i>ulib/ .wat</i> per core module with marshalling from reconstructed signatures (externs.cbor, ADR-0016)
Concurrency story	event loop, workers	event loop, worker_threads	processes + OTP supervision — the raison d'être	Tasks/threads available; designed role is a <i>leaf service</i> , not coordinator	the host's (browser/Node); fully sandboxed
Native niche	DOM/UI (Halogen, react bindings)	servers, CLIs, tooling	soft-realtime distributed systems, live supervision trees	numerics: hot kernels in Julia FFI leaves (DiffEq, DynamicalSystems, Catlab), PS as typed thin skin	portable sandboxed compute; hot kernels <i>same habitat as JS</i> — the performance
Library coverage	entire registry	entire registry	large curated package sets	prelude/effect/console/arrays/st/strings/foldable - traversable/integers/numbers/unfoldable/enums; Regex stubbed	curated <i>ulib</i> subset (Array/Eq/Ord/Show/Foldable/Functor/Iterator/...) (ADR-000)
Perf shape	V8 JIT is excellent for closure-heavy code	same	not numeric; superb latency/IO	curried-Dict glue ~325 ns/iter (measured); numerics belong behind the FFI seam	fastest of three on every benchmark; 5x allocation/pattern-match-heavy, ~1.6x (steady-state, post-warmup)
In this ecosystem	Hylograph showcases, Halogen apps	HTTPurple APIs, build tooling	purel-tidal: TidalCycles scheduling on BEAM	Hylograph compute leaves (Marginalia 219)	newly cloned (<i>purescript-backend-wasm</i>) candidate matrix column; benchmark ready to adopt

Reading the family

The interesting pattern: each non-JS backend exists to borrow a *runtime virtue* the JS engines don't have, while keeping PureScript's type system as the lingua franca.

- **purel** borrows BEAM's process model — supervision, distribution, soft-realtime scheduling. Its semantic divergences (bignum Int, binary strings) are *upgrades along the BEAM grain*, accepted rather than papered over.
- **Jurist** borrows Julia's numeric stack — the JIT, the array ecosystem, DiffEq/DynamicalSystems/Catlab. Its divergences (Int64, codepoint strings) follow the same philosophy: take the host's better number and string types, document the seam, and prove everything else identical with a differential suite.
- **purescript-backend-wasm** is the novel case: it borrows a *substrate* virtue (Wasm GC structs, sandboxing, portability) and competes **in the same habitat as JS** — beating V8's JS output on V8 itself (5–8x on allocation-heavy benchmarks) via representation: unboxed scalars, struct-subtyped ADTs, eliminated dictionaries. The first family member whose differentiation is *performance*, not habitat.
- The JS backend's virtue is *being everywhere* — and being the semantic reference the others measure against.

The same lens will apply to the additions: Go (static binaries, goroutines), Python (ubiquity, data tooling), Racket (macros and language-building).

For the comparison site (separate project)

- Columns to add: *purescript-native* (Go), *purescript-python-new*, *purescript-racket* (when it lands).
- Each cell wants a footnote link to evidence — for Jurist, most rows link to *test-suite/* results or README sections.
- A "run the same program" strip — one small PS module, its output and timing on every backend — would make the table falsifiable, in the spirit of the differential suite.